개발원가 War Game — 완전 설명서

> 반도체/디스플레이 제조업 개발원가(COP) 시뮬레이션 교육 플랫폼
> Version 0.4.0 | 2026-04-10

---

1. 개요

개발원가 War Game은 반도체/디스플레이 제조업의 원가 구조(COP: Cost of Product)를 인터랙티브하게 학습할 수 있는 교육용 웹 애플리케이션입니다.

핵심 기능: - **시뮬레이션**: 6가지 원가 변동 시나리오를 슬라이더로 조작하며 실시간 원가 변동 확인 - **War Game**: 2~6팀이 3라운드 팀 대항전으로 원가 의사결정 능력 경쟁 - **AI 코칭**: Claude API 기반 실시간 전략 코칭 (API 키 없이도 템플릿 모드 동작) - **전략 카드**: 라운드마다 전략적 선택지 제공 (투자, 구매, 인력, 품질 등)

기술 스택: Python FastAPI + Vanilla JavaScript + Chart.js + SQLite + WebSocket

---

2. 시작하기

2.1 설치 및 실행

```
# 의존성 설치
pip install fastapi uvicorn

# (선택) AI 코칭 활성화
pip install anthropic
export ANTHROPIC_API_KEY="sk-ant-..."

# 서버 시작
```

```
cd projects/cost-sim-wargame
python3 run.py
```

브라우저에서 **http://localhost:8000** 접속.

2.2 환경 설정

환경변수	기본값	설명
`HOST`	`0.0.0.0`	바인딩 주소
`PORT`	`8000`	포트
`RELOAD`	`true`	개발 모드 핫리로드
`LOG_LEVEL`	`INFO`	로그 레벨
`DB_PATH`	`data/wargame.db`	SQLite 경로
`ANTHROPIC_API_KEY`	(비어있음)	AI 코칭용 Claude API 키
`MAX_TEAMS`	`6`	최대 팀 수
`ROUND_DURATION`	`180`	라운드 제한시간(초)
`GAME_TTL_HOURS`	`24`	게임 데이터 보존 시간

3. 앱 구조 (3개 탭)

탭 1: 시뮬레이션

개별 학습용. 6가지 원가 변동 케이스를 슬라이더로 조작하며 Reference와 비교.

흐름: 1. 케이스 선택 (1~6) 2. 슬라이더 조정 → 실시간 API 호출 (디바운스 150ms) 3. Before/After 핵심 지표 비교 카드 4. 차트 시각화 (Bar chart) 5. 상세 결과 테이블 (19개 항목) 6. AI 코칭 메시지

탭 2: War Game

팀 대항전. 3라운드 경쟁으로 원가 의사결정 능력을 겨룸.

흐름: 1. **로비**: 팀 등록 (2~6팀) → 게임 규칙 확인 → 시작 2. **카운트다운**: 3 → 2 → 1 → GO! 3. **라운드 (×3회)**: - 시나리오 브리핑 + 과제 표시 - 전략 카드 3장 중 1장

선택 (선택사항) - 슬라이더 조정 + 실시간 영향도 미리보기 - 토큰 사용: 엿보기(2⊠, 방어(1⊠, AI 질문(1⊠ - 의사결정 제출 → 영향 분석 리플 표시 - 전 팀 제출 시 → AI 분석 + 라운드 드라마 - 30초 전략 논의 시간 4. **최종 결과**: 드라마틱 공개 (최하위 →1위 순서) + confetti

탭 3: 학습

참조용. COP 구조 도넛 차트, 9개 핵심 공식, 6가지 케이스 요약 카드.

---

4. 원가 계산 모델

4.1 원가 구조 (COP Structure)

```
Price (판가)
 └─ COP (제품원가)
     ├─ COM (제조원가)
     │    ├── 소요재료비 = Σ(공정별 BOM / 해당공정~최종 누적수율)
     │    └── 가공비 = Panel(노무비+경비+상각비) + Module(노무비+경비+상각비)
     └─ SGA (판관비) = 직접개발비 + 운반비 + 사업부비용 + 업무혁신 + Corporate OH
```

4.2 핵심 공식

공식	계산
누적수율	TFT수율 × CF수율 × Cell수율 × Module수율
소요재료비	Σ(공정 BOM / 해당공정 이후 누적수율)
COM	소요재료비 + 가공비
COP	COM + SGA
영업이익	Price - COP
영업이익률	영업이익 / Price × 100
Cash Cost	COP - 총 상각비
EBITDA	Price - Cash Cost
변동비	소요재료비 + 운반비

공식	계산
한계이익	Price - 변동비

4.3 기본 파라미터 (Reference)

항목	값	항목	값
Price	$200	Loading	70%
TFT 수율	99%	CF 수율	100%
Cell 수율	95%	Module 수율	97.2%
BOM TFT	$6.0	BOM CF	$5.0
BOM Cell	$1.5	BOM Module	$75.0
Panel 노무비	$21.3	Panel 경비	$11.5
Panel 상각비	$16.2	Module 노무비	$8.7
Module 경비	$5.3	Module 상각비	$7.5
SGA 합계	$28.4		

5. 6가지 시뮬레이션 케이스

Case 1: Loading 변화 (가동률)

상황: 고객사 주문 감소로 공장 가동률이 70% → 50%로 하락.

원리: 가공비는 고정비 성격 → Loading 하락 시 단위당 가공비 상승 - 가공비 변화 비율 = 기존 Loading / 신규 Loading - 예: 70%→50% = 1.4배 가공비 증가

슬라이더: Loading (20%~100%)
학습 목표: 고정비 배분 원리

Case 2: 인건비 변화

상황: 임금 인상 협상으로 인건비 0.5X~3.0X 변동.

원리: - 가공비 중 노무비(Panel+Module)가 직접 증가 - SGA 중 인건비 비중 (~30%)도 연동 상승

슬라이더: 인건비 배수 (0.5X~3.0X)
학습 목표: 인건비의 파급 효과

Case 3: 한계이익률 목표 역산

상황: 영업팀이 60% 한계이익률 목표를 제시. 필요한 재료비 절감액 역산.

원리: - 목표 한계이익 = Price × 목표율 - 허용 변동비 = Price - 목표 한계이익 - 필요 소요재료비 = 허용 변동비 - 운반비 - 재료비 절감 필요액 = 현재 소요재료비 - 필요 소요재료비

슬라이더: 목표 한계이익률 (40%~80%)
학습 목표: 목표 역산 기법

Case 4: 재료비/수율 Trade-off

상황: Module 재료비 5% 절감 가능하나 수율 4%p 하락 위험.

원리: - BOM 재료비 ↓ → 소요재료비 ↓ - 수율 ↓ → 소요재료비 ↑ (불량으로 인한 재료 낭비) - 두 효과의 크기 비교가 핵심

슬라이더: 재료비 변화(-20%~+20%), 수율 변화(-10%p~+5%p)
학습 목표: Trade-off 정량 분석

Case 5: 면취수/Mask 변화

상황: 기판당 패널 면취수 25→29개로 증가, Mask 6→7매 변경.

원리: - 면취수 ↑ → Panel BOM 및 가공비 ↓ (비율: 구 면취수/신 면취수) - Mask ↑ → Panel 가공비 ↑ (비율: 신 Mask/구 Mask)

슬라이더: 면취수 (20~35개), Mask (4~10매)
학습 목표: 공정 설계 최적화

Case 6: T/T + 투자비

상황: Module Tact Time 1.2X 지연 + 설비투자 상각비 +$1.9.

원리: - T/T 증가 → Module 가공비(노무비+경비+상각비) 비례 증가 - 투자 → 상각비 직접 증가

슬라이더: Tact 배수 (1.0X~2.0X), 투자 상각비 ($0~$5.0)
학습 목표: 설비투자 의사결정

---

6. War Game 상세 규칙

6.1 라운드별 시나리오

라운드	제목	난이도	핵심 과제
R1	Loading 하락 위기	★☆☆	Loading 50%에서 흑자 유지
R2	재료비 vs 수율 Trade-off	★★☆	재료비 절감과 수율 하락의 최적점
R3	복합 위기 상황	★★★	판가 인하 + 원자재 상승 + 공정 개선

6.2 점수 체계

순위	점수
1위	10점
2위	7점
3위	4점
4위 이하	1점

- 3라운드 누적 점수로 최종 순위 결정
- 힌트 사용 시 해당 라운드 점수 20% 감점

6.3 전략 카드 (라운드당 3장 제공, 1장 선택 가능)

카드	아이콘	효과	교육 포인트
공정혁신 투자	☒	가공비 -8%, 상각비 +$2	투자-상각비 trade-off
대량구매 계약	☒	BOM -10%, Loading 고정	규모의 경제 vs 유연성
인력 재배치	☒	노무비 -15%, 수율 -2%p	인건비-품질 trade-off

카드	아이콘	효과	교육 포인트
라인 증설	⚡	Loading +15%p, 상각비 +$3	설비투자 의사결정
품질 캠페인	☒	수율 +3%p, 경비 +$1.5	품질비용 개념
공급선 다변화	☒	서프라이즈 면역, 재료비 +3%	리스크 관리 비용

6.4 토큰 시스템

팀당 게임 전체 **5개 토큰** 보유. 특수 행동에 소모:

행동	비용	효과
☒경쟁팀 엿보기	2☒	다른 팀의 현재 영업이익 확인
☒서프라이즈 방어	1☒	이번 라운드 서프라이즈 이벤트 무효화
☒AI 코칭 질문	1☒	AI에게 전략 조언 요청

6.5 서프라이즈 이벤트 (R3 전용)

이벤트	효과
환율 급등	수입 재료비 3% 상승
경쟁사 가격 공세	판가 -$10
공정 혁신 성공	TFT 수율 +1%p
에너지 비용 상승	Panel 경비 15% 증가
신규 고객 확보	Loading +10%p

6.6 라운드 드라마

상황	연출
역전 (리더 교체)	☒confetti + shake 애니메이션
접전 ($3 이내)	⚡ 긴장감 사운드 + 파란 강조
압도적 리드 ($15+)	☒강조 메시지

7. 실시간 피드백 시스템

7.1 영향도 미리보기 (슬라이더 조정 시)

슬라이더 + 전략 카드 변경 시 **클라이언트 측 즉시 계산**으로 서버 호출 없이 표시: - 예상 영업이익 $X.X (녹색: 흑자 / 빨간: 적자) - 재료비 $X.X | 가공비 $X.X | SGA $X.X

7.2 영향 분석 리플 (제출 직후)

서버에서 각 파라미터별 영업이익 기여도 계산: - Loading 변경 → $-5.2 - 재료비 변경 → $+2.1 - 수율 변경 → $-1.8 - 전략 카드 → $+1.5

가장 큰 영향 항목이 시각적으로 강조됨.

---

8. AI 코칭

8.1 두 가지 모드

모드	조건	특징
AI 모드	`ANTHROPIC_API_KEY` 설정됨	Claude Sonnet 4.6 기반 동적 응답
템플릿 모드	API 키 없음	사전 작성된 9개 키워드 매칭 응답

8.2 AI 채팅 (하단 패널)

질문 가능 키워드: 수율, 가공비, 재료비, COP, 면취수, Mask, Tact, 인건비, 한계이익

8.3 게임 중 AI 코칭

라운드 중 "AI 코치에게 전략 질문" 입력란에서 시나리오 맥락을 반영한 힌트 제공. - AI 모드: 시나리오 + 과제 + 학습목표를 컨텍스트로 받아 사고를 유도 - 템플릿 모드: 라운드별 맞춤 힌트 제공

---

9. 교육 운영 가이드

9.1 추천 수업 진행 순서

단계	시간	활동
1. 이론 강의	20분	COP 구조, 핵심 공식 설명 (학습 탭 활용)
2. 개별 실습	15분	시뮬레이션 탭에서 Case 1~6 순서대로 체험
3. 팀 구성	5분	2~6팀 편성, 팀명 등록
4. War Game	30분	3라운드 경쟁 (라운드당 3분 + 분석 + 전략시간)
5. 종합 토론	15분	최종 결과 리뷰, 학습 포인트 정리

9.2 진행자 팁

- **R1 전**: "가공비가 왜 고정비인지" 간단히 설명
- **R2 전**: "BOM 절감이 항상 좋은 건 아니다" 질문 유도
- **R3 전**: "실제 현업에서는 여러 변수가 동시에 움직인다" 강조
- 서프라이즈 이벤트는 R3에서 진행자가 직접 트리거 (`/api/game/{id}/surprise`)
- 전략 카드에 대해 "왜 그 카드를 선택했는지" 팀별 발표 유도

9.3 LAN 멀티플레이

같은 네트워크의 여러 기기에서 접속 가능:

```
서버 PC: python3 run.py
참가자: http://서버IP:8000 접속
```

---

10. API 레퍼런스

10.1 시뮬레이션 API

메서드	엔드포인트	파라미터	설명
GET	`/api/reference/{case_num}`	case_num: 1~6	기준값 조회
POST	`/api/simulate/loading`	loading: 0.01~1.0	Case 1
POST	`/api/simulate/labor`	multiplier: 0.1~10.0	Case 2
POST	`/api/simulate/marginal-profit`	target_rate: 0.01~0.99	Case 3
POST	`/api/simulate/material-yield`	material_change_pct, module_yield_change	Case 4
POST	`/api/simulate/cuts-mask`	new_cuts: 1~100, new_mask: 1~20	Case 5
POST	`/api/simulate/tact-investment`	tact_multiplier, investment_delta	Case 6

10.2 게임 API

메서드	엔드포인트	설명
POST	`/api/game/create`	게임 생성 (team_names, total_rounds)
GET	`/api/game/{game_id}`	게임 상태 조회
POST	`/api/game/{game_id}/start-round`	라운드 시작
POST	`/api/game/{game_id}/submit`	의사결정 제출 (params, card_id)

메서드	엔드포인트	설명
POST	`/api/game/{game_id}/surprise`	서프라이즈 이벤트 트리거
GET	`/api/game/{game_id}/leaderboard`	리더보드 조회
POST	`/api/game/{game_id}/spend-token`	토큰 사용 (peek/shield)
POST	`/api/game/{game_id}/ask-ai`	라운드 중 AI 코칭
GET	`/api/game/{game_id}/history`	게임 이력 조회
GET	`/api/games`	최근 게임 목록

10.3 기타 API

메서드	엔드포인트	설명
GET	`/api/health`	서버 상태 (version, uptime, active_games, ai_enabled)
POST	`/api/chat`	AI 채팅
WS	`/ws/game/{game_id}`	게임 실시간 동기화

11. 프로젝트 구조

```
cost-sim-wargame/
├── run.py                  # 서버 시작 스크립트
├── api/
│   ├── main.py              # FastAPI 서버 (v0.4.0)
│   ├── config.py            # 환경 설정
│   └── database.py          # SQLite 영속성
├── engine/
```

```
│   └── cost_model.py        # 원가 계산 엔진 (27개 검증)
├── ai/
│   └── assistant.py         # AI 코칭 + 시나리오 + 전략 카드
├── frontend/
│   ├── index.html           # SPA HTML
│   ├── css/
│   │   ├── style.css        # @import 진입점
│   │   ├── tokens.css       # CSS 변수
│   │   ├── animations.css   # 애니메이션
│   │   ├── base.css         # 기본 스타일
│   │   ├── components.css   # UI 컴포넌트
│   │   ├── game.css         # War Game 스타일
│   │   ├── chat.css         # 채팅 패널
│   │   └── responsive.css   # 반응형
│   └── js/
│       ├── main.js          # ES Module 진입점
│       └── modules/
│           ├── state.js     # 상태 관리
│           ├── api.js       # API 래퍼
│           ├── simulation.js # 시뮬레이션
│           ├── wargame.js   # War Game 로직
│           ├── chart.js     # 차트
│           ├── ui.js        # Toast/Modal/Loading
│           ├── audio.js     # 사운드
│           ├── chat.js      # AI 채팅
│           ├── onboarding.js # 온보딩
│           └── utils.js     # 유틸리티
├── tests/
│   ├── test_engine.py       # 엔진 테스트 (24개)
│   ├── test_api.py          # API 테스트 (23개)
│   └── capture.py           # 스크린샷 캡처
└── data/
    └── wargame.db           # SQLite (자동 생성)
```

---

12. 테스트

```
# 전체 테스트 (47개)
python3 -m pytest tests/ -v

# 엔진만
python3 -m pytest tests/test_engine.py -v

# API만
```

```
python3 -m pytest tests/test_api.py -v

# 인라인 엔진 검증 (27개)
python3 engine/cost_model.py
```

---

13. 문제 해결

증상	해결
AI 배지가 "Template"으로 표시	`ANTHROPIC_API_KEY` 환경변수 확인
게임 데이터가 사라짐	서버 재시작 시 인메모리 세션 초기화됨. SQLite에 이력은 보존
포트 충돌	`PORT=8001 python3 run.py` 로 다른 포트 사용
모바일에서 줌 안 됨	의도된 동작 (앱 같은 경험을 위해 `user-scalable=no` 설정)
WebSocket 연결 끊김	자동 재연결 (exponential backoff, 최대 10회)